

CSE 1201

Hashing

Hashing

Searching Methods:

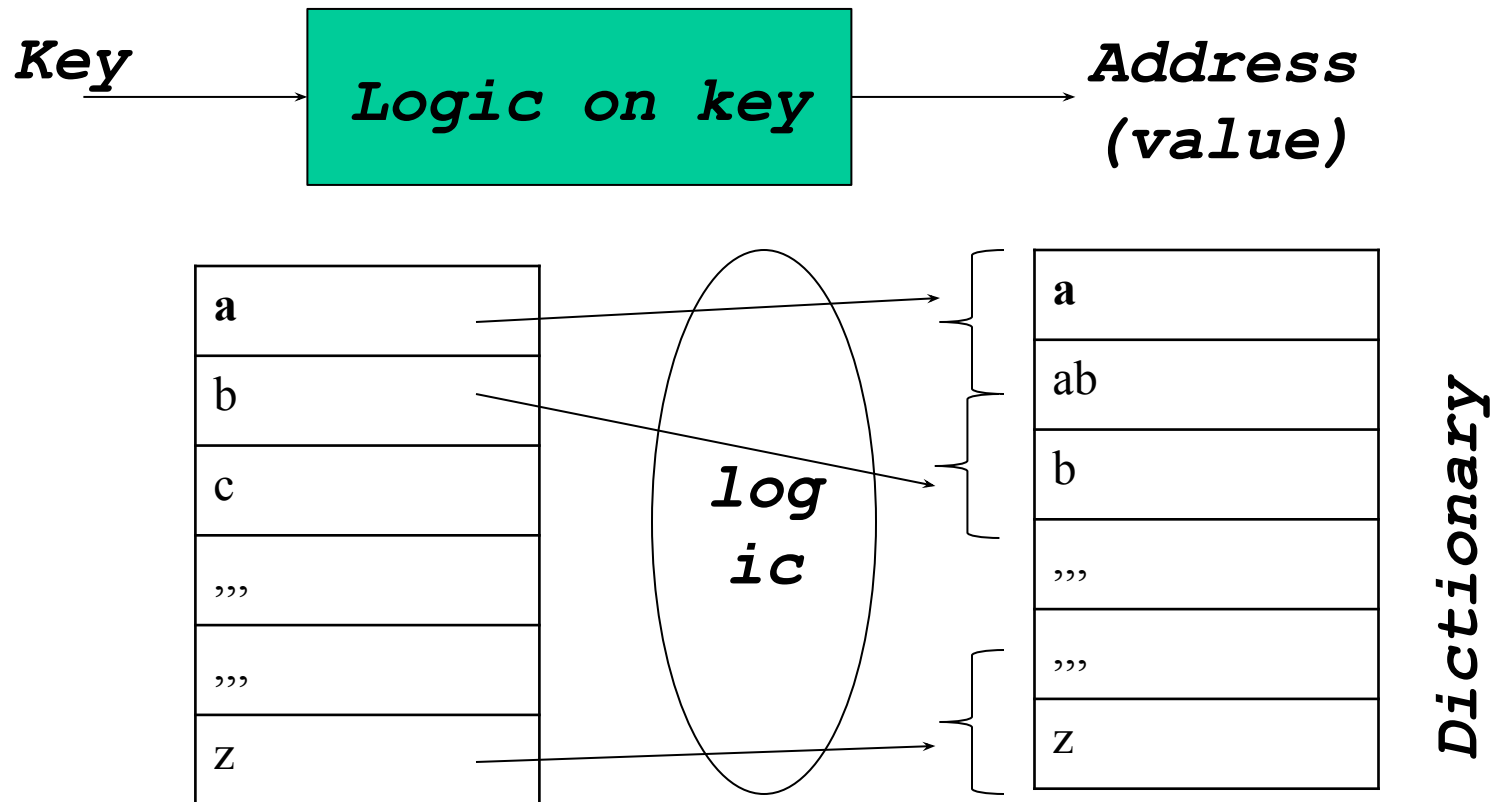
1. Linear Search – $O(n)$

12	6	7	7	9	4	56	55	34
----	---	---	---	---	---	----	----	----

2. Binary Search – $O(\log n)$

12	6	7	7	9	4	56	55	34
----	---	---	---	---	---	----	----	----

Hashing: It is a searching technique that relay on the concept of Hash Table.



Hashing:

- Fast Searching: The objective is to search in $O(1)$
- Fast insertion/removal

Hash Table: It is data structure that associates key and value.

Key	Value

To perform searching using hashing we need a function $H(k)$ that one-to-one mapping to integers in $(0, L-1)$

L: size of array

H(k) : Hash of key k

Properties of good Hash function:

- fast to compute
- Uniform distribution of data

index	Roll (Key)	Value
0	12	Rahim
1	34	Karim
2	54	Aslam
3	99	Jabbar

Example

Keys – 8, 3, 13, 6, 4, 10

<i>A[]</i>															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Properties of good Hash function:

- fast to compute
- Uniform distribution of data

index	Roll (Key)	Value
0	12	Rahim
1	34	Karim
2	54	Aslam
3	99	Jabbar

Example

Keys – 8, 3, 13, 6, 4, 10

A[]				3	4		6		8		10			13	
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Insert k=8

Insert k=10

Insert k=12

No data

Example

Keys - 8, 3, 13, 6, 4, 10

A[]

			3	4		6		8		10			13	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Insert k=50 ??

No space to store 50

Need to increase the size of A[]

Lot of memory is wasted

*So to improve the technique, we need a
Mathematical model for Hash function*

Example

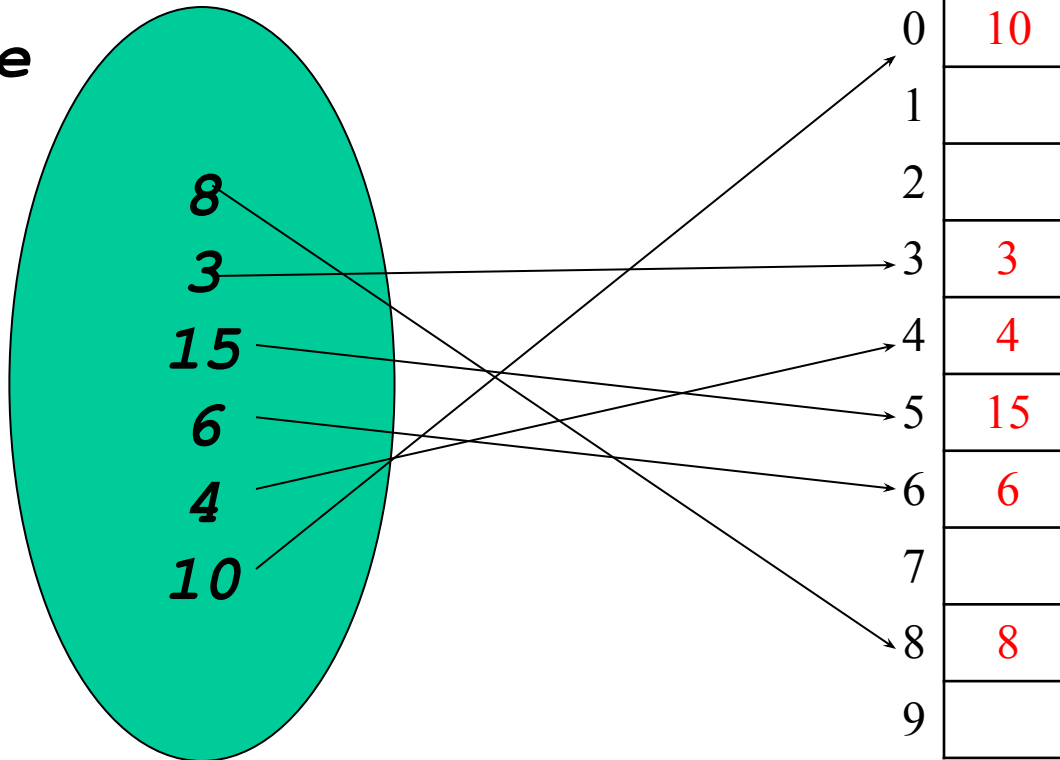
Keys - 8, 3, 15, 6, 4, 10

Hash Table

Key space

$$H(k) = k \% \text{size}$$

$$H(8) = 8 \% 10 \\ = 8$$



Example

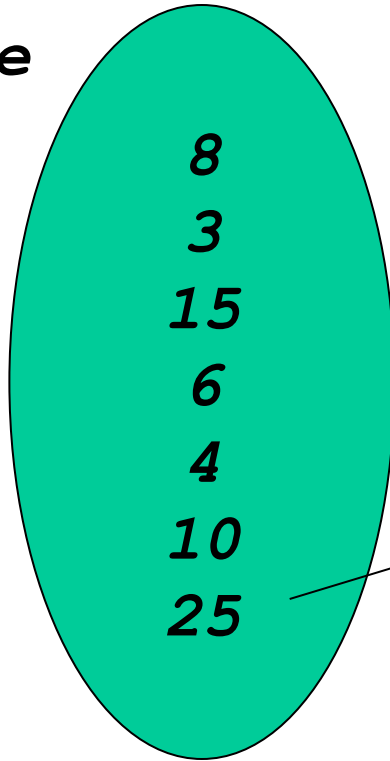
Keys - 8, 3, 15, 6, 4, 10, 25

Hash Table

Key space

$$H(k) = k \% \text{size}$$

$$H(25) = 25 \% 10 \\ = 5$$



0	10	
1		
2		
3	3	
4	4	
5	15,25	collision
6	6	
7		
8	8	
9		

Collision Resolution Methods

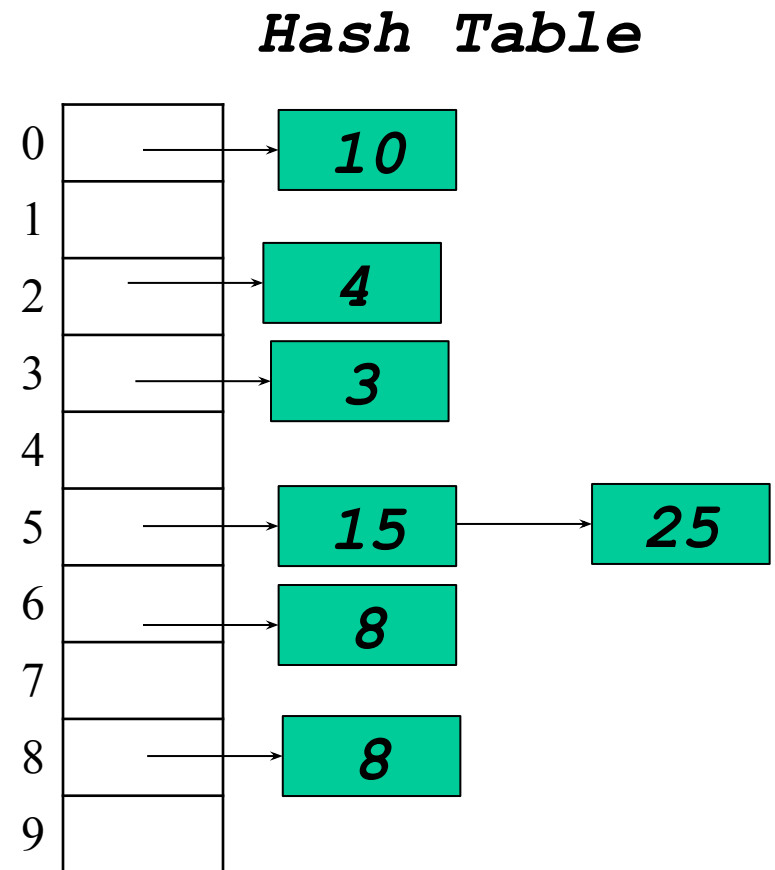
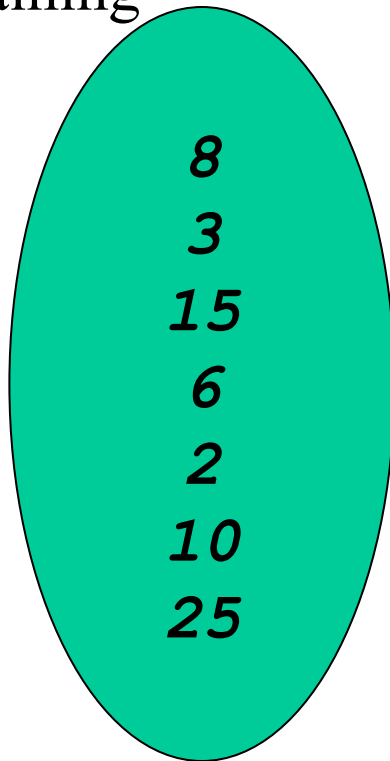
1. Open Hashing
Chaining
2. Closed Hashing
 - Linear Probing
 - Quadratic Probing

Hash Table

0	10	<i>collision</i>
1		
2		
3	3	
4	4	
5	15,25	
6	6	
7		
8	8	
9		

Collision Resolution Methods

1. Open Hashing Chaining



Collision Resolution Methods

2. Closed Hashing

- Linear Probing

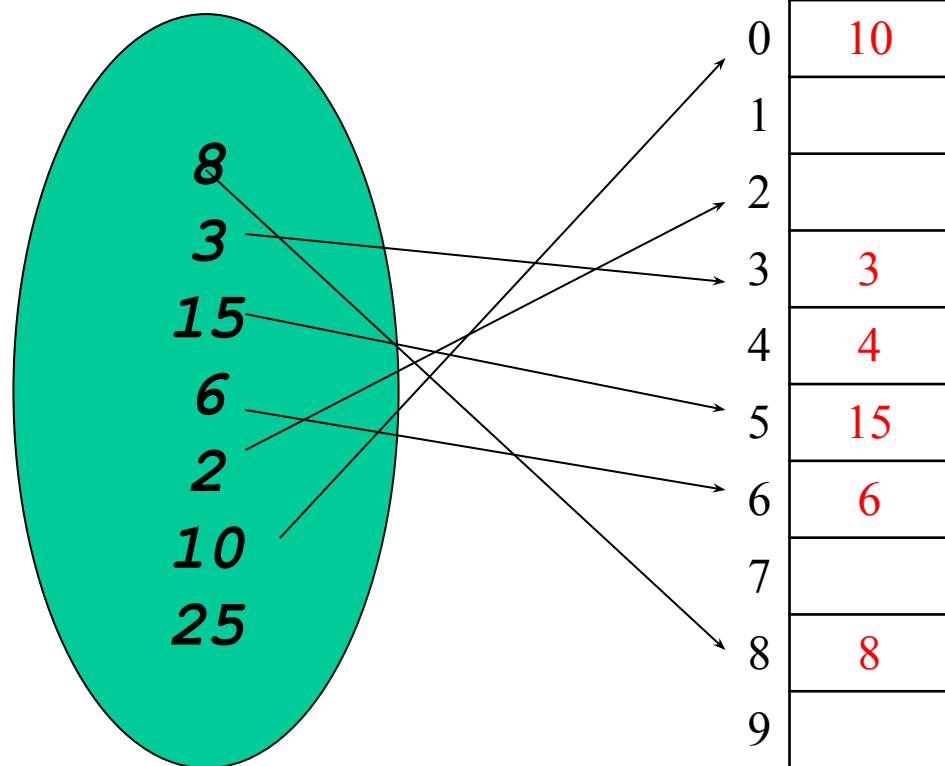
$$H'(x) = [H(k) + f(i)] \% \text{size}$$

$$f(i) = i$$

$$i = 0, 2, 3, 4, \dots$$

$$\begin{aligned} H(8) &= [H(8) + f(0)] \% 10 \\ &= [8 + 0] \% 10 = 8 \end{aligned}$$

Hash Table



Collision Resolution Methods

2. Closed Hashing

- Linear Probing

$$H'(x) = [H(k) + f(i)] \% \text{size}$$

$$f(i) = i$$

$$i = 0, 2, 3, 4, \dots$$

$$\begin{aligned} H(25) &= [H(25) + f(0)] \% 10 \\ &= [25 + 0] \% 10 = 5 \end{aligned}$$

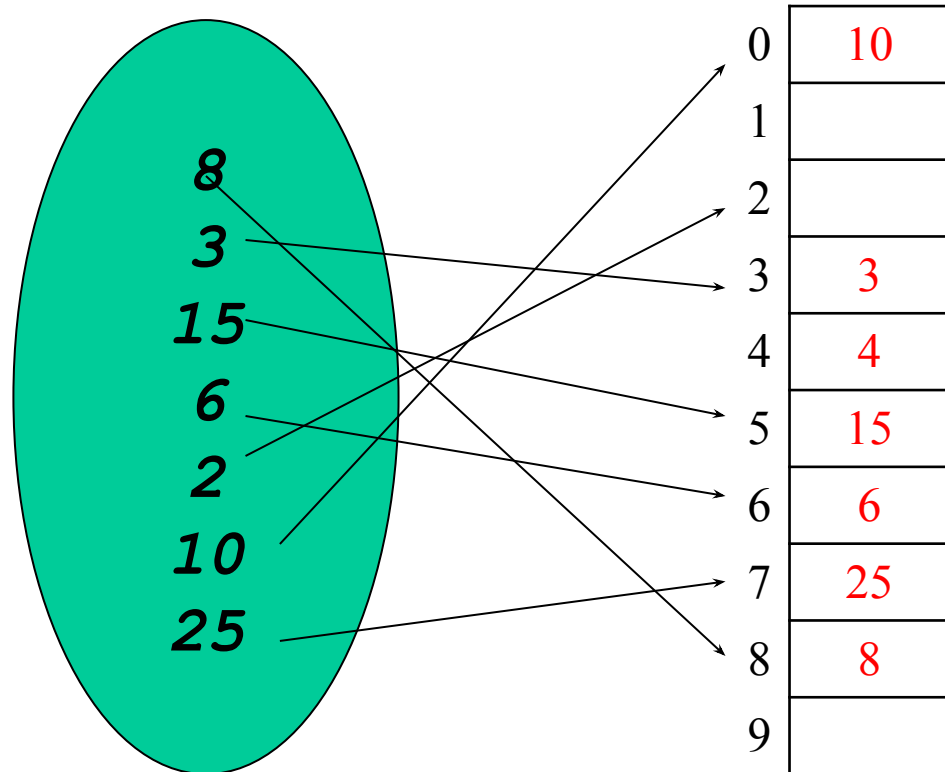
collision

$$\begin{aligned} H(25) &= [H(25) + f(1)] \% 10 \\ &= [25 + 1] \% 10 = 6 \end{aligned}$$

Occupied

$$\begin{aligned} H(25) &= [H(25) + f(2)] \% 10 \\ &= [25 + 2] \% 10 = 7 \end{aligned}$$

Hash Table



Collision Resolution Methods

2. Closed Hashing

- Quadratic Probing

$$H'(x) = [H(k) + f(i)] \% \text{size}$$

$$f(i) = i^2$$

$$i = 0, 2, 3, 4, \dots$$

$$\begin{aligned} H(25) &= [H(25) + f(0)] \% 10 \\ &= [25 + 0^2] \% 10 = 5 \end{aligned}$$

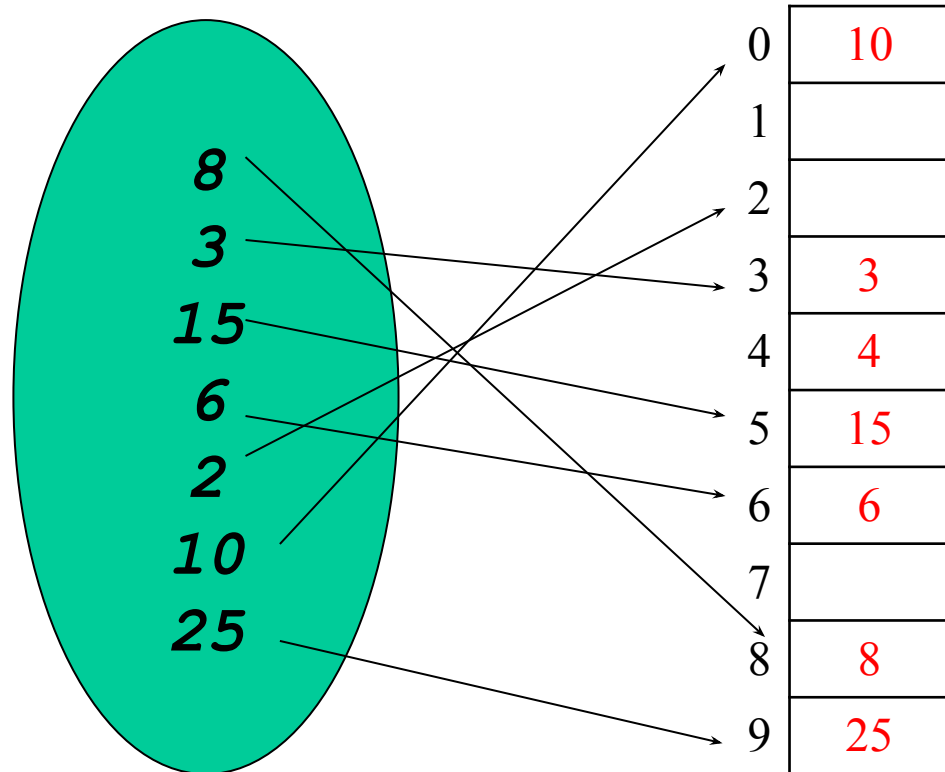
collision

$$\begin{aligned} H(25) &= [H(25) + f(1)] \% 10 \\ &= [25 + 1^2] \% 10 = 6 \end{aligned}$$

Occupied

$$\begin{aligned} H(25) &= [H(25) + f(2)] \% 10 \\ &= [25 + 2^2] \% 10 = 9 \end{aligned}$$

Hash Table



String as a key

- Sum the ASCII value of all characters in a string

Example

Mia (M=77, i=105 , a=97)

Sum = 77+105+97 = 279

$$\begin{aligned} H(297) &= [H(279) + f(0)] \% 11 \\ &= [279 + 0] \% 11 = 4 \end{aligned}$$

Hash Table

0	
1	
2	
3	
4	Mia
5	
6	
7	
8	
9	
10	

String as a key

- Sum the ASCII value of all characters in a string

Example

Tim (T=84, i=105 , a=109

Sum = 84+105+109 = 298

$H(298) = [H(298) + f(0)] \% 11$
 $= [298 + 0] \% 11 = 1$

Hash Table

0	
1	Tim
2	
3	
4	Mia
5	
6	
7	
8	
9	
10	

String as a key

- Sum the ASCII value of all characters in a string

Example

Now find Tim (T=84, i=105 , a=109

Sum = 84+105+109 = 298

$$\begin{aligned} H(298) &= [H(298) + f(0)] \% 11 \\ &= [298 + 0] \% 11 = 1 \end{aligned}$$

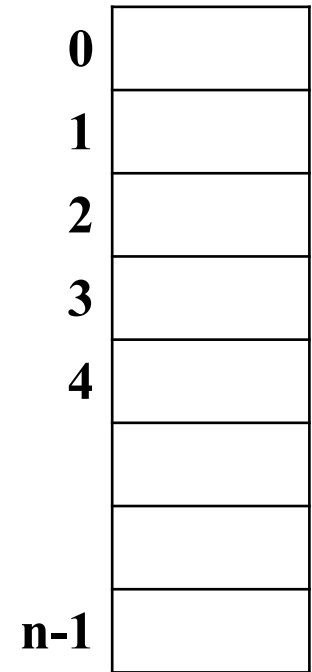
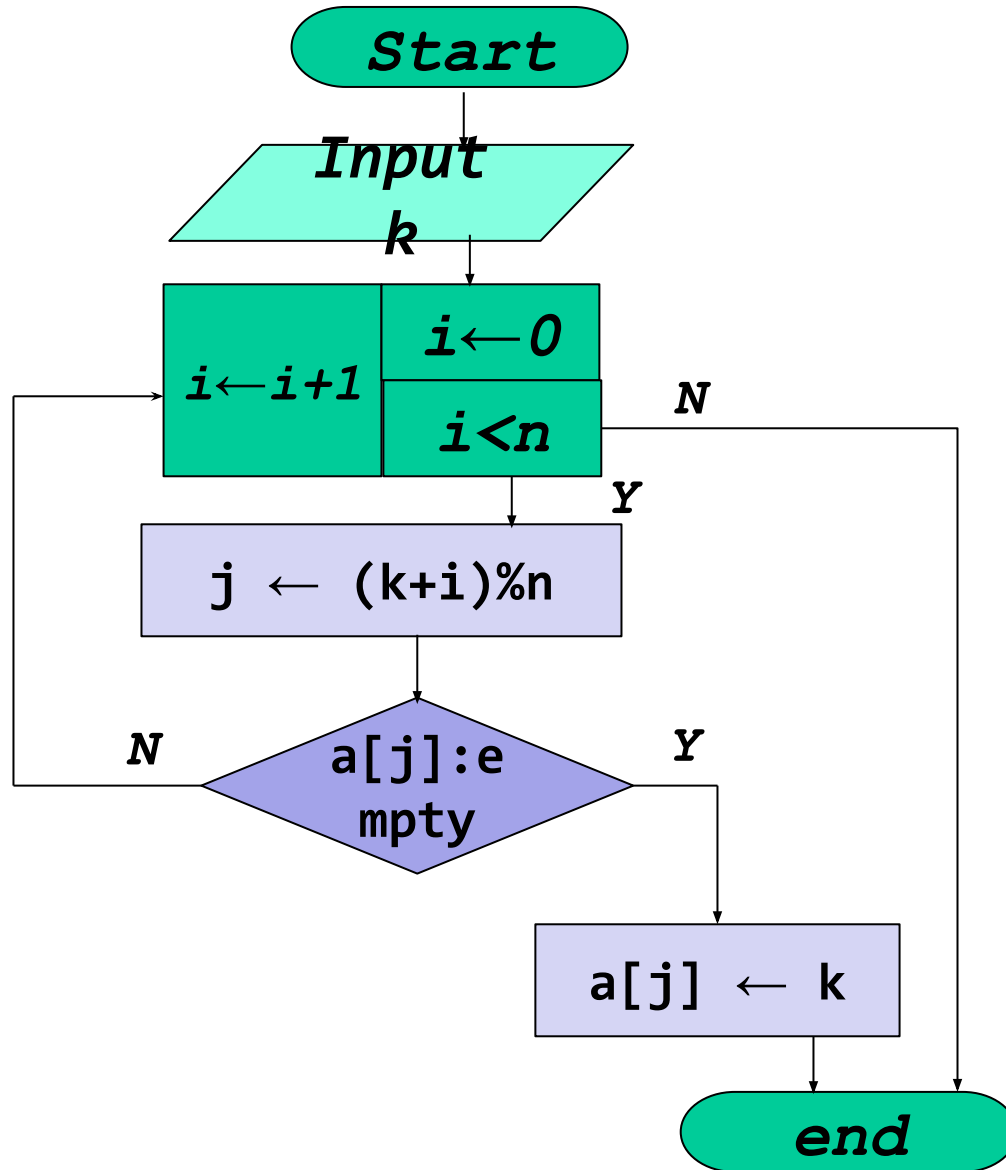
Hash Table

0	Bea
1	Tim
2	Leo
3	Sam
4	Mia
5	Zoe
6	Jan
7	Lou
8	Max
9	Ada
10	Ted

Hashing Algorithm

- Calculation applied to a key to transform it into an address
- For numeric keys, divide the key by the number of available addresses, n , and take the remainder, *$address = key \text{ Mod } n$*
- For alphanumeric keys, divide the sum of ASCII codes in a key by the number of available addresses, n , and take the remainder.
- Folding method divides key into equal parts then adds parts together
 - The mobile number 01452 8345654, becomes $01+45+28+34+56+54=218$ and find $H(218)$

Algorithm for Hashing with Linear Probing

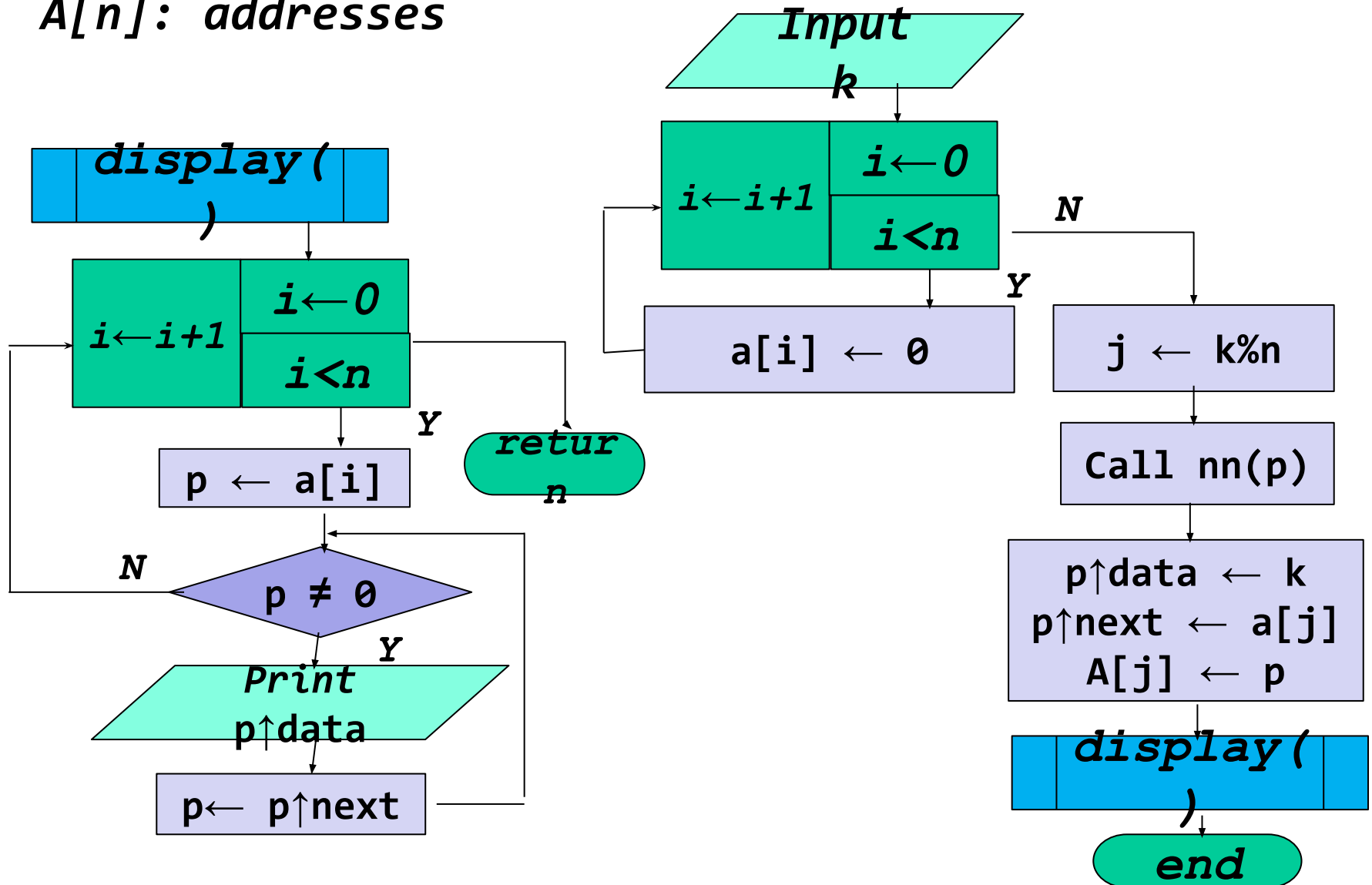


$A[n]$

k : new element

Algorithm for Hashing with Chaining

k : new element
 $A[n]$: addresses



Deletion problem

- $H = \text{KEY} \bmod 10$
- Insert 47, 57, 68, 18, 67
- Find 68
- Find 10
- Delete 47
- Find 57

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Deletion Problem -- SOLUTION

- “Lazy” deletion
- Each cell is in one of 3 possible states:
 - active
 - empty
 - deleted
- For Find or Delete
 - only stop search when EMPTY state detected (not DELETED)